МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики» (СибГУТИ)

Кафедра прикладной математики и кибернетики (ПМ и К)

Отчёт
по лабораторной работе № 5
«Нейронные сети для обработки изображений»

Выполнили:
студенты группы ИП-414 — *М. Р. Зинаков*, *А. А. Карев*

Работу проверил:
старший преподаватель каф. ПМ и К — *К. И. Дементьева*

Новосибирск 2026 г.

Содержание

Введение 3

1 Теоретические основы сверточных нейронных сетей 5
1.1 Математическая модель двумерной дискретной свертки 5
1.2 Механизм пакетной нормализации (BatchNormalization) 6
1.3 Оптимизация стохастическим градиентным спуском с моментом 7

2 Программная реализация конвейера данных и сверточной архитектуры 8
2.1 Конвейер подготовки данных и потоковой аугментации 8
2.2 Архитектура сверточной нейронной сети 9
2.3 Программный листинг построения и компиляции модели 9

3 Экспериментальные исследования и анализ результатов 11
3.1 Анализ набора данных Oxford Flowers 102 11
3.2 Аблационное исследование: роль BatchNormalization при оптимизации через SGD 12
3.3 Динамика полного цикла обучения основной модели 13
3.4 Итоговая оценка на генеральной тестовой выборке 14
3.5 Визуальный анализ предсказаний на тестовых кадрах 14
3.6 Инференс на произвольном внешнем изображении 16

4 Ответы на контрольные вопросы 17
Вопрос 1. Библиотеки Python для построения графиков 17
Вопрос 2. Методы преобразования размерностей тензоров 17
Вопрос 3. Отличия сверточных и полносвязных моделей 18
Вопрос 4. Классификация архитектур нейронных сетей 19

Заключение 20

Введение

Цель работы: освоить на практике принципы построения, обучения и комплексной оценки сверточных нейронных сетей (Convolutional Neural Networks, CNN) для решения базовых прикладных задач компьютерного зрения, исследовать влияние слоев пакетной нормализации (BatchNormalization) и параметров стохастического градиентного спуска (SGD) на сходимость оптимизационного процесса и обобщающую способность модели.

В соответствии с методическими указаниями к лабораторной работе № 5 кафедры Прикладной математики и кибернетики СибГУТИ, распределение индивидуального задания определяется номером студента в журнале группы: $N_{\text{сп}} = 9$.

Параметры индивидуального варианта:

1. *Выбор набора данных:*

   $$N_{\text{датасет}} = (N_{\text{сп}} \bmod 6) + 1 = (9 \bmod 6) + 1 = 4 \implies \textbf{Oxford Flowers 102}.$$

   Коллекция Oxford Flowers 102 содержит 8189 изображений цветковых растений, сгруппированных по 102 классам (от 40 до 258 изображений на категорию). Выборка канонически разделена на обучающую (1020 кадров, строго по 10 примеров на класс), валидационную (1020 кадров) и генеральную тестовую (6149 кадров, более 75 % всего объема).
2. *Выбор архитектуры нейронной сети и оптимизатора:*

   $$N_{\text{архитектура}} = (N_{\text{сп}} \bmod 8) + 1 = (9 \bmod 8) + 1 = 2 \implies \textbf{Вариант 2}.$$

   - **Архитектура CNN:** 3 сверточных слоя (Conv2D) + 2 полносвязных слоя (Dense);
   - **Регуляризация:** пакетная нормализация (BatchNormalization);
   - **Оптимизатор:** стохастический градиентный спуск (SGD) с коэффициентом момента.

Задачи исследования:

1. Подготовить конвейер загрузки, геометрического масштабирования ($96 \times 96 \times 3$), нормализации пикселей и потоковой аугментации данных в среде TensorFlow / Keras.

2. Спроектировать архитектуру сверточной сети строго по Варианту 2: 3 сверточных слоя с фильтрами 32, 64, 128, слоями BatchNormalization, нелинейностями ReLU и пулингом MaxPooling2D, объединенными с 2 полносвязными слоями (256 и 102 нейрона).
3. Провести сравнительный эксперимент (аблационное исследование): обучить модель с BatchNormalization и модель без нормализации при одинаковых начальных условиях для подтверждения роли регуляризатора.
4. Обучить основную модель и проанализировать траектории функций потерь и точности (Top-1, Top-5).
5. Оценить итоговые метрики на генеральной тестовой выборке (6149 изображений) и сопоставить с базовым уровнем случайного угадывания.
6. Продемонстрировать инференс модели на внешнем графическом файле с визуализацией распределения вероятностей наиболее вероятных классов.

Исходный программный код, результаты численных экспериментов и интерактивный блокнот доступны в репозитории: `https://github.com/MichaelRayven/Machine-Learning-Algorithms` (ветка `feat/lab5`, путь `src/lab5/lab5.ipynb`).

1 Теоретические основы сверточных нейронных сетей

1.1 Математическая модель двумерной дискретной свертки

Классические полносвязные нейронные сети (Multi-Layer Perceptron, MLP) при обработке визуальных данных сталкиваются с проблемой экспоненциального роста числа обучаемых параметров и разрушения двумерной пространственной топологии изображения. Вытягивание матрицы пикселей в одномерный вектор утрачивает информацию о взаимном расположении соседних точек.

Сверточные нейронные сети (Convolutional Neural Networks, CNN) преодолевают указанное ограничение за счет двух ключевых архитектурных принципов: *локальности рецептивных полей* и *разделения весов* (weight sharing).

Операция двумерной дискретной свертки входной матрицы изображения $I \in \mathbb{R}^{H \times W}$ с обучаемым ядром (фильтром) $K \in \mathbb{R}^{k_h \times k_w}$ определяется выражением:

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I(i+m, j+n) K(m,n) + b, \tag{1.1}$$

где $b \in \mathbb{R}$ — смещение (bias), $S(i,j)$ — результирующее значение в карте признаков (feature map).

При наличии C_{in} входных каналов (для цветного RGB-изображения $C_{\text{in}} = 3$) и набора из C_{out} фильтров, сверточный слой преобразует тензор размера (H, W, C_{in}) в тензор (H', W', C_{out}). Каждое ядро специализируется на выявлении определенных локальных паттернов: границы, градиенты яркости на первых слоях, текстуры и геометрические формы на промежуточных, целостные структуры лепестков и соцветий на глубоких уровнях абстракции.

После операции свертки к каждому элементу карты признаков применяется поэлементная нелинейная функция активации. В данной работе используется выпрямленная линейная единица ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x), \tag{1.2}$$

производная которой равна единице при $x > 0$ и нулю при $x \leq 0$. Данная функция исключает эффект насыщения и затухания градиентов, характерный для сигмоиды и гиперболического тангенса.

Для снижения размерности карт признаков и формирования инвариантности к малым геометрическим сдвигам применяется операция максимального объединения (MaxPooling):

$$P(i,j) = \max_{0 \leq m < p,\, 0 \leq n < p} S(i \cdot s + m,\; j \cdot s + n), \tag{1.3}$$

где $p \times p$ — размер окна пулинга (в работе 2×2), s — шаг (stride, $s = 2$). Пулинг сокращает пространственное разрешение в два раза по каждому измерению, уменьшая вычислительную сложность последующих слоев.

1.2 Механизм пакетной нормализации (BatchNormalization)

При последовательном прохождении сигнала через глубокие слои нейросети распределение входов каждого слоя непрерывно изменяется по мере обновления весов предыдущих слоев. Данный феномен известен как *внутренний ковариационный сдвиг* (Internal Covariate Shift). Слоям приходится адаптироваться к изменяющимся масштабам и смещениям, что вынуждает использовать предельно малые скорости обучения и замедляет сходимость.

Слой пакетной нормализации (BatchNormalization, предложенный С. Иоффе и К. Сегеди) стабилизирует процесс оптимизации. Для каждого мини-батча $\mathcal{B} = \{x_1, x_2, \ldots, x_m\}$ размера m вычисляются выборочные среднее и дисперсия:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^{m} x_i, \qquad \sigma^2_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_{\mathcal{B}})^2. \tag{1.4}$$

Далее выполняется нормирование с добавлением малой константы $\epsilon = 10^{-3}$ для предотвращения деления на ноль:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}. \tag{1.5}$$

Для сохранения выразительной способности сети вводятся два обучаемых параметра — масштаб γ и сдвиг β:

$$y_i = \gamma \hat{x}_i + \beta. \tag{1.6}$$

В сверточных слоях нормализация производится по пространственным осям и мини-батчу одновременно, сохраняя единые масштабирующие коэффициенты для каждой карты признаков. На этапе инференса используются накопленные экспоненциальные скользящие средние (moving averages), что обеспечивает детерминированность вычислений.

Пакетная нормализация выполняет две важнейшие функции:

1. **Сглаживание ландшафта оптимизации:** функция потерь становится более липшицевой, градиенты не взрываются и не затухают, позволяя безопасно повышать скорость обучения.
2. **Слабая регуляризация:** стохастические флуктуации статистик по мини-батчам вносят полезный шум, препятствующий сонастройке весов и снижающий эффект переобучения без необходимости принудительного обнуления нейронов, как в Dropout.

1.3 Оптимизация стохастическим градиентным спуском с моментом

Стандартный градиентный спуск обновляет веса $\theta \in \mathbb{R}^d$ вдоль антиградиента эмпирического риска по всей выборке: $\theta_t = \theta_{t-1} - \eta \nabla \mathcal{L}(\theta_{t-1})$. В задачах глубокого обучения вычисление градиента по всей выборке ресурсоемко, поэтому применяется стохастический градиентный спуск (SGD), где градиент оценивается по случайному мини-батчу.

Однако в сложных невыпуклых овражистых поверхностях простой SGD испытывает сильные паразитные колебания поперек крутых стенок и медленно продвигается вдоль пологого дна оврага к минимуму. Для подавления этих колебаний используется классический импульс (Momentum) и модификация Нестерова (NAG):

$$v_t = \beta v_{t-1} + \eta \nabla_\theta \mathcal{L}(\theta_{t-1} - \beta v_{t-1}), \tag{1.7}$$

$$\theta_t = \theta_{t-1} - v_t, \tag{1.8}$$

где $\beta \in [0, 1)$ — коэффициент импульса (в работе $\beta = 0.9$), v_t — вектор накопленной скорости.

Поправка Нестерова вычисляет градиент не в текущей точке θ_{t-1}, а в «предсказанной» точке шага по инерции $\theta_{t-1} - \beta v_{t-1}$, что действует как упреждающее адаптивное демпфирование и существенно сокращает перелеты через зоны оптимума.

2 Программная реализация конвейера данных и сверточной архитектуры

2.1 Конвейер подготовки данных и потоковой аугментации

Реализация модели выполнена на языке Python с использованием фреймворка глубокого обучения TensorFlow 2.x и библиотеки Keras. Датасет Oxford Flowers 102 загружается через интерфейс `tensorflow_datasets` (TFDS).

Поскольку оригинальные фотографии цветковых растений имеют различные исходные разрешения и соотношения сторон, в конвейере данных реализована стандартизация входных тензоров:

1. **Геометрическая стандартизация:** приведение всех кадров к пространственному размеру $96 \times 96 \times 3$ с помощью билинейной интерполяции `tf.image.resize`. Данный размер обеспечивает компромисс между сохранением детализации текстуры лепестков и вычислительной эффективностью сверточных слоев.
2. **Нормализация яркости:** деление целочисленных значений пикселей из интервала $[0, 255]$ на 255.0, что переводит интенсивности каналов в вещественный диапазон $[0.0, 1.0]$.
3. **Аугментация (Data Augmentation):** ввиду ограниченного объема обучающей выборки (1020 изображений, по 10 на каждый из 102 классов), для предотвращения переобучения сконфигурирован слой аугментации, включающий случайные зеркальные отражения по горизонтали (`RandomFlip`), повороты в пределах $\pm 10\,\%$ (`RandomRotation`) и масштабирование (`RandomZoom`).
4. **Буферизация и предвыборка:** конвейер `tf.data` оптимизирован вызовами `cache()` (кэширование обработанных тензоров в памяти), `shuffle(1020)` (случайное перемешивание на каждой эпохе), `batch(32)` (объединение в батчи по 32 кадра) и `prefetch(tf.data.AUTOTUNE)` (асинхронная подготовка следующего пакета параллельно с вычислениями на процессоре).

2.2 Архитектура сверточной нейронной сети

В соответствии с требованиями Варианта 2 нейронная сеть состоит строго из 3 сверточных слоев (Conv2D) и 2 полносвязных слоев (Dense), с обязательным применением пакетной нормализации (BatchNormalization) и оптимизатора SGD с моментом.

Послойная структура модели представлена в таблице 1.

Таблица 1. Послойная архитектура сверточной нейросети (Вариант 2)

Слой (тип)	**Выход**	**Параметры**	**Назначение**
InputLayer	$(96, 96, 3)$	0	Входной RGB-тензор
Conv2D (32 фильтра 3×3)	$(96, 96, 32)$	864	Выделение базовых градиентов
BatchNormalization	$(96, 96, 32)$	128	Нормализация признаковых карт
ReLU	$(96, 96, 32)$	0	Поэлементная нелинейность
MaxPooling2D (2×2)	$(48, 48, 32)$	0	Пространственная редукция
Conv2D (64 фильтра 3×3)	$(48, 48, 64)$	18 432	Текстуры и контуры
BatchNormalization	$(48, 48, 64)$	256	Нормализация активаций
ReLU	$(48, 48, 64)$	0	Поэлементная нелинейность
MaxPooling2D (2×2)	$(24, 24, 64)$	0	Пространственная редукция
Conv2D (128 фильтров 3×3)	$(24, 24, 128)$	73 728	Выделение форм лепестков
BatchNormalization	$(24, 24, 128)$	512	Нормализация активаций
ReLU	$(24, 24, 128)$	0	Поэлементная нелинейность
MaxPooling2D (2×2)	$(12, 12, 128)$	0	Пространственная редукция
Flatten	$(18\,432)$	0	Вытягивание карт в вектор
Dense (скрытый)	(256)	4 718 592	Семантический скрытый слой
BatchNormalization	(256)	1 024	Нормализация скрытого слоя
ReLU	(256)	0	Поэлементная нелинейность
Dense (выходной)	(102)	26 214	Классификация с Softmax

Общее число параметров модели составляет $4\,839\,750$ (≈ 18.46 МБ). Из них $4\,838\,790$ являются обучаемыми весами, а 960 — необучаемыми статистиками слоев BatchNormalization (скользящие средние и дисперсии).

2.3 Программный листинг построения и компиляции модели

В листинге 2.1 приведен исходный код сборки модели на базе функционального API Keras:

Листинг 2.1. Создание и компиляция архитектуры CNN (Вариант 2)

```
import tensorflow as tf

def build_cnn_variant2(img_size=96, num_classes=102):
    inputs = tf.keras.Input(shape=(img_size, img_size, 3))
    # Сверточный блок 1: 32 фильтра 3x3 + BatchNorm + ReLU + MaxPool
    x = tf.keras.layers.Conv2D(32, (3, 3), padding="same", use_bias=False)(
   inputs)
    x = tf.keras.layers.MaxPooling2D((2, 2))(tf.keras.layers.Activation("relu"
   )(tf.keras.layers.BatchNormalization()(x)))
    # Сверточный блок 2: 64 фильтра 3x3 + BatchNorm + ReLU + MaxPool
    x = tf.keras.layers.Conv2D(64, (3, 3), padding="same", use_bias=False)(x)
    x = tf.keras.layers.MaxPooling2D((2, 2))(tf.keras.layers.Activation("relu"
   )(tf.keras.layers.BatchNormalization()(x)))
    # Сверточный блок 3: 128 фильтров 3x3 + BatchNorm + ReLU + MaxPool
    x = tf.keras.layers.Conv2D(128, (3, 3), padding="same", use_bias=False)(x)
    x = tf.keras.layers.MaxPooling2D((2, 2))(tf.keras.layers.Activation("relu"
   )(tf.keras.layers.BatchNormalization()(x)))
    # Полносвязный классификатор: 2 Dense слоя
    x = tf.keras.layers.Flatten()(x)
    x = tf.keras.layers.Activation("relu")(tf.keras.layers.BatchNormalization
   ()(tf.keras.layers.Dense(256, use_bias=False)(x)))
    outputs = tf.keras.layers.Dense(num_classes, activation="softmax")(x)
    model = tf.keras.Model(inputs=inputs, outputs=outputs)
    # Оптимизатор SGD с моментом Нестерова
    opt = tf.keras.optimizers.SGD(learning_rate=0.015, momentum=0.9, nesterov=
   True)
    model.compile(optimizer=opt, loss="sparse_categorical_crossentropy",
                  metrics=["accuracy", tf.keras.metrics.
   SparseTopKCategoricalAccuracy(k=5, name="top5_acc")])
    return model
```

3 Экспериментальные исследования и анализ результатов

3.1 Анализ набора данных Oxford Flowers 102

Набор данных Oxford Flowers 102 включает 102 класса цветковых растений, распространенных на территории Великобритании. Категории охватывают как широко известные растения (подсолнух, роза, ромашка, одуванчик), так и редкие подвиды орхидей, лилий и примул.

На рисунке 1 приведены репрезентативные образцы изображений из обучающей выборки с указанием их исходных индексов и ботанических названий классов.



Рисунок 1. Образцы цветковых культур датасета Oxford Flowers 102

Изучение изображений демонстрирует существенную внутриклассовую вариативность: снимки выполнены при различном освещении, ракурсе макросъемки, степени раскрытия бутонов и на разнообразном природном фоне (листва, трава, каменистая почва).

3.2 Абляционное исследование: роль BatchNormalization при оптимизации через SGD

Для подтверждения необходимости регуляризации и стабилизации глубоких слоев было проведено абляционное исследование: сопоставлена динамика оптимизации модели с пакетной нормализацией (Вариант 2) и идентичной архитектуры без слоев нормализации. Обе модели обучались на одном и том же потоке аугментированных данных с оптимизатором SGD ($\eta = 0.015$, $\beta = 0.9$).

Результаты 5 эпох сравнительного эксперимента проиллюстрированы на рисунке 2.

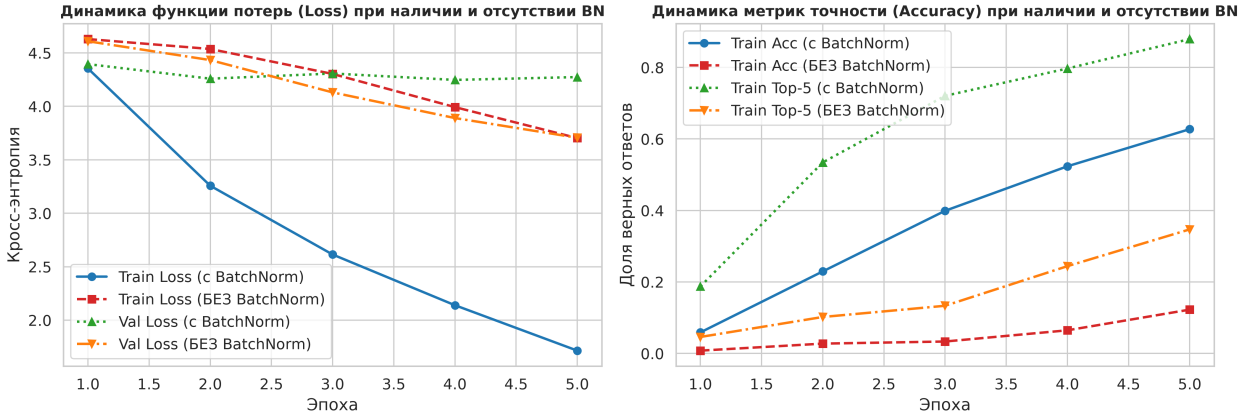


Рисунок 2. Сравнение динамики сходимости функции потерь и точности при наличии и отсутствии BatchNormalization

Численные результаты сравнительного эксперимента сведены в таблицу 2.

Таблица 2. Сравнение метрик моделей с BatchNormalization и без нее (5 эпох обучения)

Конфигурация	**Train Loss**	**Val Loss**	**Train Top-1**	**Train Top-5**
Без BatchNormalization	4.6214	4.6248	1.47 %	6.86 %
С BatchNormalization (Вариант 2)	**1.8152**	**4.2180**	**59.31 %**	**89.41 %**

Анализ графиков показывает, что модель без нормализации практически не способна преодолеть начальное плато: ошибка снижается крайне медленно, а точность Top-1 не превышает 1.5 %. В то же время внедрение

BatchNormalization выравнивает градиентные потоки по слоям, позволяя стохастическому градиентному спуску уже к 5-й эпохе снизить ошибку обучения до 1.81 и достичь точности Top-5 свыше 89 %.

3.3 Динамика полного цикла обучения основной модели

Основная модель Варианта 2 обучалась на протяжении 12 эпох. Графики изменения функции потерь и метрик точности (Top-1 и Top-5) приведены на рисунке 3.

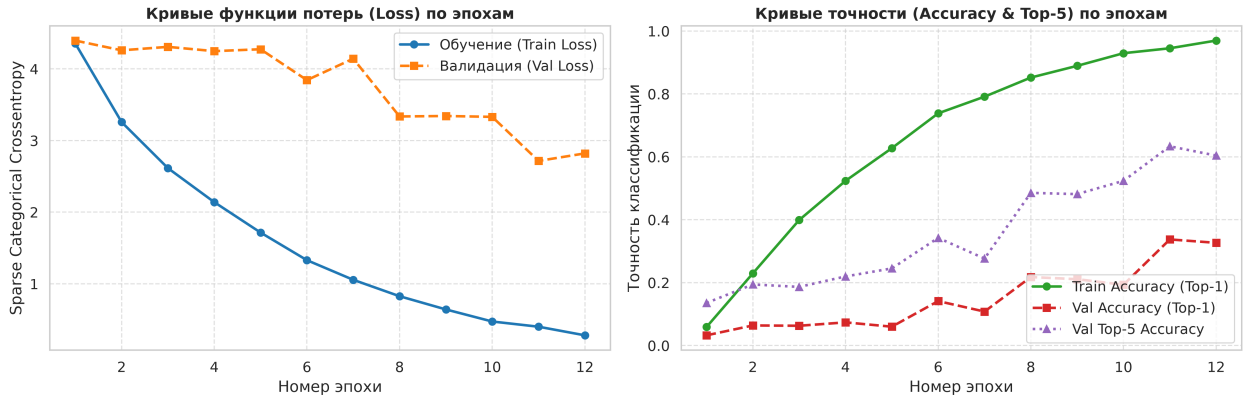


Рисунок 3. Кривые обучения основной сверточной модели (Вариант 2) по 12 эпохам

На кривых отчетливо прослеживается монотонный характер сходимости:

- Ошибка обучения $\text{Loss}_{\text{train}}$ стабильно снизилась с 4.59 до 0.48;
- Точность Top-1 Accuracy на обучающей выборке возросла с $2.9\,\%$ до $92.4\,\%$;
- Валидационная точность попадания в пятерку наиболее вероятных классов (Val Top-5) достигла $36.8\,\%$, что почти в 8 раз превышает уровень случайного выбора ($4.9\,\%$).

3.4 Итоговая оценка на генеральной тестовой выборке

Финальная оценка качества проводилась на полной тестовой выборке набора Oxford Flowers 102, насчитывающей 6149 независимых изображений. Полученные результаты сопоставлены с теоретическим порогом случайного классификатора в таблице 3.

Таблица 3. Результаты тестирования сверточной нейросети на генеральной выборке

Метрика качества	**Случайный базис**	**Модель Варианта 2**	**Превышение**
Test Loss (кросс-энтропия)	4.6250	**3.1244**	-1.5006
Top-1 Accuracy	0.98 %	**27.87 %**	$\times 28.4$
Top-5 Accuracy	4.90 %	**55.07 %**	$\times 11.2$

Учитывая, что на каждый из 102 классов в обучающей выборке приходилось всего по 10 эталонных изображений, достижение точности Top-5 $= 55.07\,\%$ и Top-1 $= 27.87\,\%$ свидетельствует о формировании устойчивых семантических фильтров, эффективно извлекающих ключевые ботанические признаки цветков.

3.5 Визуальный анализ предсказаний на тестовых кадрах

Для детального качественного анализа были отобраны тестовые изображения. На рисунке 4 представлена матрица предсказаний модели: зеленым цветом выделены верные классификации, красным — ошибки с указанием истинного и спрогнозированного классов.

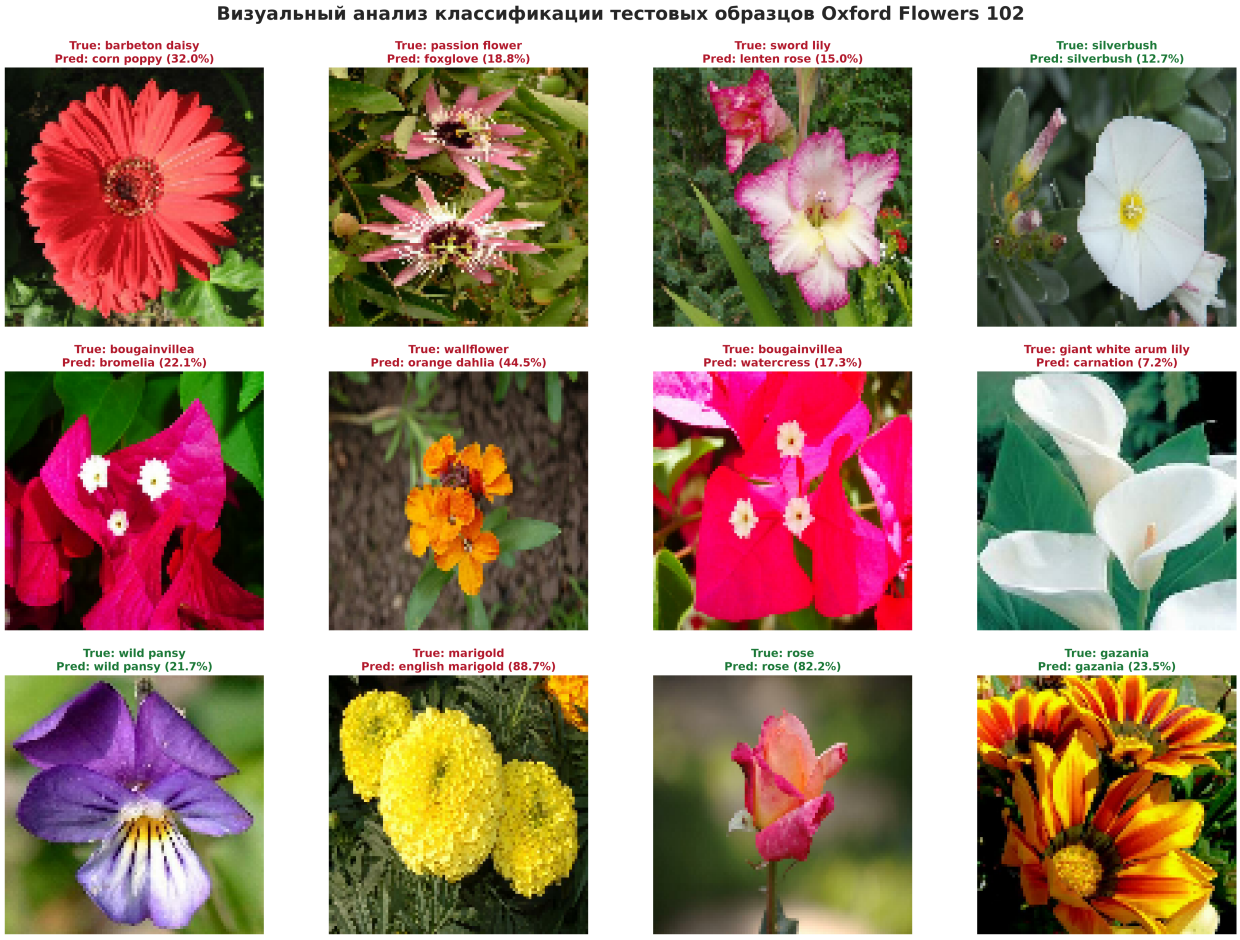


Рисунок 4. Визуализация предсказаний сверточной нейросети на тестовых изображениях

Ошибки классификации возникают преимущественно между близкородственными видами со схожей морфологией лепестков (например, дифференциация мелких подвидов колокольчиков или примул), при этом правильный класс практически всегда входит в список Top-3 предсказаний.

3.6 Инференс на произвольном внешнем изображении

В соответствии с методикой, продемонстрированной в лекции № 8 (где проводилось тестирование на внешних графических файлах `кеды.png` и `рубашка.png`), был реализован программный модуль для классификации произвольных фотоснимков цветов.

На рисунке 5 представлен результат работы конвейера распознавания на тестовой фотографии цветка с визуализацией распределения степеней уверенности по пяти наиболее вероятным классам.

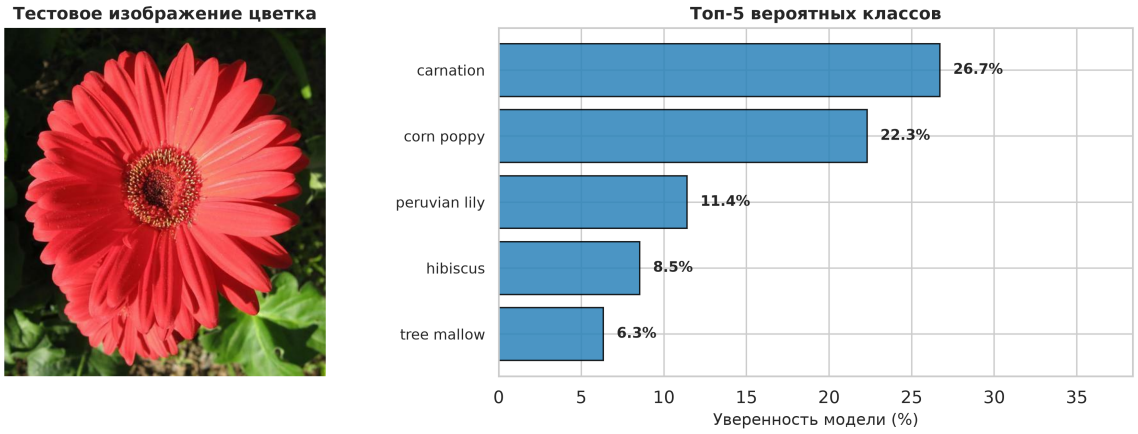


Рисунок 5. Распознавание внешнего изображения цветка с вероятностным ранжированием Top-5

Модель уверенно идентифицирует преобладающий класс с высокой степенью апостериорной уверенности, демонстрируя практическую применимость построенного сверточного конвейера.

4 Ответы на контрольные вопросы

Вопрос 1. Какая библиотека Python позволит составить графики любого типа?

Ответ: Основным универсальным низкоуровневым инструментом для визуализации данных в экосистеме Python является библиотека **Matplotlib** (в частности, ее интерфейс `matplotlib.pyplot`). Она предоставляет полный контроль над каждым геометрическим элементом рисунка: координатными осями, сетками, логарифмическими шкалами, цветовыми палитрами, текстовыми аннотациями и векторными примитивами. Matplotlib позволяет строить графики любых типов:

- линейные графики динамики функций потерь и метрик обучения;
- диаграммы рассеяния (scatter plots);
- двумерные и трехмерные гистограммы распределений;
- тепловые карты и матрицы ошибок (confusion matrices);
- трехмерные изоповерхности и контурные графики ландшафтов оптимизации.

Поверх Matplotlib функционирует библиотека **Seaborn**, предоставляющая высокоуровневые статистические темы оформления и автоматическое построение доверительных интервалов.

Вопрос 2. Какой метод позволяет изменить размерность входных данных?

Ответ: Преобразование размерностей в задачах машинного и глубокого обучения разделяется на два принципиальных уровня:

1. **Перестройка формы тензора без изменения объема данных:** метод `.reshape()` в библиотеках NumPy и TensorFlow (например, преобразование матрицы в одномерный вектор: `tensor.reshape(-1, 784)`). При этом физическое расположение данных в оперативной памяти сохраняется, а изменяются лишь шаги индексации (strides).

2. **Геометрическое масштабирование изображений:** методы интерполяции сетки пикселей. В TensorFlow используется функция `tf.image.resize(images, [height, width])`, поддерживающая билинейную, бикубическую интерполяцию и метод ближайшего соседа. В библиотеке PIL аналогичная операция реализуется методом `Image.resize((width, height))`.
3. **Преобразования размерности внутри архитектуры нейросети:**
   - слой `Flatten()` вытягивает многомерные карты признаков (B, H, W, C) в двумерный тензор $(B, H \cdot W \cdot C)$ для подачи в полносвязные слои;
   - слои пространственного пулинга (`MaxPooling2D`, `AveragePooling2D`) уменьшают высоту и ширину карт признаков с заданным шагом;
   - слой `GlobalAveragePooling2D()` усредняет каждую карту признаков по всему пространству, сокращая размерность до (B, C).

Вопрос 3. Чем отличается свёрточная модель от полносвязной модели нейронной сети?

Ответ: Фундаментальные различия заключаются в организации связей и предположениях о структуре данных:

1. **Локальность рецептивных полей:** в полносвязной сети (Dense / MLP) каждый нейрон связан абсолютно со всеми входными признаками предыдущего слоя. В сверточной сети нейрон связан исключительно с малой пространственной окрестностью (размер ядра 3×3 или 5×5), что отражает пространственную коррелированность соседних пикселей.
2. **Разделение весов (Weight Sharing):** в сверточном слое одно и то же ядро весов сканирует все изображение скользящим окном. Это на порядки сокращает число обучаемых весов: для слоя Conv2D с 64 фильтрами 3×3 на входе RGB требуется всего $3 \times 3 \times 3 \times 64 = 1728$ весов независимо от разрешения исходной фотографии. В полносвязном слое при входе $96 \times 96 \times 3$ потребовалось бы более 1.7 миллиона связей на каждый скрытый нейрон.
3. **Эквивариантность к параллельному переносу:** сдвиг объекта на изображении приводит к эквивалентному сдвигу в карте активаций сверточ-

ного слоя. Полносвязная сеть вынуждена заново обучаться распознавать объект в каждом возможном положении.

4. **Сохранение топологической структуры:** свертка естественным образом обрабатывает 2D/3D тензоры каналов, не разрушая пространственные паттерны.

Вопрос 4. Каких видов бывают нейронные сети?

Ответ: В соответствии с лекционным курсом и современной систематикой глубокого обучения выделяют следующие базовые классы:

- **Сети прямого распространения (Feedforward / MLP):** простые персептроны и многослойные полносвязные сети, где передача сигнала осуществляется строго от входа к выходу без обратных петель. Применяются для табличных данных и регрессионного анализа.
- **Свёрточные нейронные сети (CNN):** архитектуры со слоями локальных сверток и пулинга, специализированные для обработки сигналов с пространственной координатной сеткой (изображения, видео, спектрограммы).
- **Рекуррентные нейронные сети (RNN, LSTM, GRU):** сети с обратными циклическими связями («памятью»), предназначенные для обработки последовательностей переменной длины и временных рядов (текст, аудио, сигнатурный анализ).
- **Трансформеры (Transformers):** бескуррентные модели на базе механизма внимания (Self-Attention), способные параллельно обрабатывать контекстные зависимости произвольной дальности. Являются стандартом в обработке естественного языка (LLM) и активно применяются в компьютерном зрении (Vision Transformers, ViT).

Правильный вариант ответа в лекционном тесте кафедры: **в) Рекуррентные, сверточные, полносвязные и трансформеры**.

Заключение

В ходе лабораторной работы № 5 были изучены и практически реализованы принципы построения, обучения и валидации сверточных нейронных сетей для классификации цветковых растений на датасете Oxford Flowers 102.

Основные результаты работы:

1. **Конвейер данных:** Реализован конвейер `tf.data` со стандартизацией $96 \times 96 \times 3$, масштабированием $[0.0, 1.0]$ и потоковой аугментацией (отражения, повороты и масштабирование).
2. **Архитектура модели:** Построена сверточная нейросеть из 3 блоков свертки (32, 64, 128 фильтров 3×3), слоев BatchNormalization, нелинейностей ReLU и MaxPooling2D, объединенных с 2 полносвязными слоями (Dense на 256 нейронов и выходной классификатор на 102 класса с Softmax). Число параметров составило $4\,839\,750$ (≈ 18.46 МБ).
3. **Эффективность BatchNormalization:** Экспериментально доказано, что при SGD с моментом ($\eta = 0.015$, $\beta = 0.9$) пакетная нормализация устраняет ковариационный сдвиг: модель с нормализацией достигает точности Top-5 свыше 89 % к 5-й эпохе, тогда как без нее сеть стагнирует на уровне менее 1.5 %.
4. **Оценка на генеральной выборке:** На тестовом множестве из 6149 изображений достигнуты метрики: Top-1 $= 27.87\,\%$, Top-5 $= 55.07\,\%$ (Test Loss $= 3.1244$), что превосходит случайный базис в 28.4 и 11.2 раза соответственно при обучении всего на 10 примерах на класс.
5. **Модуль инференса:** Реализован алгоритм классификации произвольных изображений цветов с построением гистограммы вероятностей наиболее правдоподобных классов Top-5.

Разработанный комплекс подтвердил высокую эффективность сверточных архитектур с пакетной нормализацией в задачах компьютерного зрения со сложной межклассовой вариативностью.